Gymnasium Oberwil

Maturarbeit

Bildsynthese von menschlichen Gesichtern mithilfe von Kuenstlicher Intelligenz

Eine Anwendung eines Autoencoders

Luis Wirth, 4a

Betreut von
Dr. Jonas Gloor

Abgegeben im September 2019

Inhaltsverzeichnis

1 Vorwort **2**

2 Einleitung **3**

3 Machinelles Lernen **4**
3.1 Daten 4
3.2 Modelle (allgemein) 4
3.3 Training 4
3.3.1 Kostenfunktionen 4
3.3.2 Gradientenverfahren (engl.: Gradient descent) 5
3.4 Neural Networks 6
3.4.1 Backpropagation 6
3.4.2 Feedforward Neural Network 6
3.4.3 Convolutional Neural Networks 6
3.4.4 Autoencoder 6
3.4.5 Variational Autoencoder? 6
3.4.6 Detangled Autoencoder? 6

4 Diskussion **7**

5 Schlusswort **8**

1 Vorwort

2 Einleitung

3 Machinelles Lernen

Der Themenbereich des Machinellen Lernens beschäftigt sich mit Algorithmen, um mathematischen Modellen beizubringen, Probleme zu loesen. Hierzu werden keine expliziten Instruktionen fest einprogrammiert, sondern das Modell wird trainiert und optimiert sich von selbst. Dabei wird das Modell mit Trainingsdaten befuettern, mithilfe dessen eine Korrelation zwischen den Inputdaten und den Outputdaten erlernt werden soll.

3.1 Daten

Der Trainingsdatensatz beinhaltet gewisse **Attribute**, die Merkmalstypen der Inputdaten. Die **Features** sind dann die spezifischen Werte der Attribute eines Trainingssample. Die sogenannten **Labels** sind die erwarteten Outputs, welche der Algorithmus vorherzusagen hat. Diese Vorhersagen werden mit den Labels abgeglichen und so bewertet. Anhand der Bewertungen wird dann eine Optimierung des Modells vorgenommen. Unter korrekten Bedingungen (keine Ueberanpassung) findet kein Auswendiglernen der Trainingsdaten statt, sondern ein Generalisieren des Zusammenhangs, anhand von Mustern und Gesetzmassigkeiten. Um eine Endgueltige Bewertung des Modells durchzufuehren, wird ein Testdatensatz, welcher nicht Teil des Trainingsdatensatz ist, genutzt um Vorhersagen zu machen. Dies garantiert, dass kein Auswendiglernen moeglich ist.

Die Inputs/Features werden in einem Vektor $\vec{x} = (x_1, x_2, ..., x_n)$ und die Outputs/Vohersagen in einem Vektor $\vec{y} = (y_1, y_2, ..., y_m)$ zusammengefasst. Das gleiche macht man mit den Labels, wobei diese mit $\vec{\hat{y}} = (\hat{y}_1, \hat{y}_2, ..., \hat{y}_m)$ bezeichnet werden.

Abbildung 1: eine Veranschaulichung eines Machine Learning Modells

3.2 Modelle (allgemein)

Ein Modell ist eine mathematische Funktion $f : \mathbb{R}^n \to \mathbb{R}^m$ welche die Inputs auf die Outputs abbildet $\vec{y} = f(\vec{x})$. Im Zusammenhang mit Wertevorhersagen spricht man von einem Regressionsmodell. Das Verhalten dieses Modells wird bestimmt durch seine **Modellparameter** $\phi_1, \phi_2, ...$ Das Ziel ist es die Modellparameter so einzustellen, dass die Vorherrsagen $\vec{y}$ besser mit den Labels $\vec{\hat{y}}$ uebereinstimmen. Dies wird iterativ gemacht, indem immer wieder leichte Anpassungen an den Paramtern vorgenommen werden, bis sie das gewunschte Resultat liefern.

Das wohl einfachste Regressionsmodell ist eine Regressionsgerade. Diese ist angemessen, falls ein einfacher linearen Zusammenhang zwischen den Features und den Labels besteht. $y = \phi_1 x + \phi_0$. Nun muessten nur noch ϕ_0 und ϕ_1 bestimmt werden.

Neben den gelernten Parametern, gibt es auch noch sogenannte **Hyperparameter**. Diese koennen nicht erlernt werden, sondern muessen manuell vor dem Training gewaehlt werden und koennen den Lernvorgang erheblich beeinflussen.

BEISPIELMODELL (in der Art: Note anhand Lernstunden vorhersagen)

3.3 Training

3.3.1 Kostenfunktionen

Einsicht ist der erste Schritt zur Besserung. Das gilt auch beim Machine Learning, deshalb muss beim Training zuerst die Genauigkeit des Models bewertet werden. Dafuer gibt es sogennante Kosten-, Verlust- oder Fehlerfunktionen. Sie soll ein Mass fuer die Abweichung des Outputs $\vec{y}$ vom Label $\vec{\hat{y}}$ sein.

Die Fehler der einzlnen Outputs c_i werden aufsummiert um den Fehler C einer gesamten Vorhersage zu berechnen.

$$C(\vec{y}, \vec{\hat{y}}) = \sum_{i=1}^{m} c(y_i, \hat{y}_i)$$

Der Fehler eines Datensatzen $\bar{C}$ ergibt sich aus dem arithmetischen Mittel der Vorhersagen-Fehler.

$$\bar{C} = \frac{1}{p} \sum_{j=1}^{p} C\left(\vec{y}_j, \vec{\hat{y}}_j\right)$$

Eine Kostenfunktion $c(y_i, \hat{y}_i)$ sollte folgende Eigenschaften aufweisen:

- c ist 0, wenn $y = \hat{y}$
- c waechst mit $|\vec{y} - \vec{\hat{y}}|$
- C ist nach jedem y_n partiell differenzierbar

Eine beliebte Kostenfunktion ist der "Mean Squared Error"

3.3.2 Gradientenverfahren (engl.: Gradient descent)

Nun gilt es ein Optimierungsproblem zu loesen, konkret: Die Fehlerfunktion $C(\vec{y}, \vec{\hat{y}})$ muss minimiert werden. Hierfuer verwendet man das sogennante **Gradientenverfahren**. [1]

Allgemein koennen mithilfe des Gradientenverfahrens Funkionen $f(x_1, x_2, \cdots, x_n)$ vom Typ $\mathbb{R}^n \to \mathbb{R}$ minimiert werden. Dies beschieht, in dem ein Startpunkt (Ortsvektor) $\vec{p}_0$, dessen Komponenten den Input von f darstellt, gewaehlt wird. Nun werden iterativ neue Punkte $\vec{p}_z$ gesucht, welche immer naeher beim lokalen Minimun liegen, also Punkte die den Funktionswert $f(\vec{p}_z)$ immer kleiner werden lassen. Dies wird durchgeführt, bis der Punkt genug nahe beim lokalen Minimun ist.

Dafuer muss ein Vektor $\vec{b}_z$ bestimmt werden, welcher auf den Punkt $\vec{p}_z$ draufaddiert einen neuen Punkt $\vec{p}_{z+1}$ bildet, bei dem der Funktionswert $f(\vec{p}_{z+1})$ kleiner ist als der von $f(\vec{p}_z)$. Dies geschieht am effizientesten, wenn $\vec{b}_z$ in die Richtung der staerksten Funktionswertabnahme zeigt.

Hierzu braucht man den sogennanten **Gradient** $\vec{\nabla}$.

Gradient

Der Gradient $\vec{\nabla}$ ist ein Differentialoperator, welcher ein Skalarfeld auf ein Vektorfeld (das sogennante Gradientenfeld) abbildet.
Der Gradient $\vec{\nabla} f(\vec{p})$ einer Funktion f bei einem Punkt $\vec{p}$ ist ein Vektor, welcher in die Richtung des steilsten Anstieg von f zeigt. Sein Magnitude ist ein Mass fuer die Starke der Zunahme. Somit ist er eine Erweiterung der Ableitung auf n-dimensionale Funktionen.

$$grad(f) = \vec{\nabla} f = \begin{pmatrix} \frac{\partial f}{\partial x_1} \\ \vdots \\ \frac{\partial f}{\partial x_2} \end{pmatrix}$$

Bei den $\frac{\partial f}{\partial x_i}$ handelt es sich um partielle Ableitungen.

[1] Im Gymnasium lernt man die lokalen Extrema (inkl. lokalen Minimas) zu bestimmen, indem man die erste Ableitung f' gleich null setzt. Dies ist hier nicht moeglich, da die Funktion C' deutlich zu komplex ist, um die Nullstellen zu bestimmen. Deshalb braucht man das Gradientenverfahren.

Partielle Ableitungen

Somit ist der Vektor $\vec{b}_z$ einfach ein skalierter negativer Gradient der Funktion f im Punkt $\vec{p}_z$. Also kann jetzt das iterative Annahern des lokalen Minimums so beschreiben werden: $\vec{p}_{z+1} = \vec{p}_z - \eta \cdot \vec{\nabla} f(\vec{p}_z)$, wobei η die Schrittgroesse festlegt.

3.4 Neural Networks

3.4.1 Backpropagation

3.4.2 Feedforward Neural Network

3.4.3 Convolutional Neural Networks

3.4.4 Autoencoder

3.4.5 Variational Autoencoder?

3.4.6 Detangled Autoencoder?

4 Diskussion

5 Schlusswort

Abbildungsverzeichnis

1 eine Veranschaulichung eines Machine Learning Modells 4

Tabellenverzeichnis

Selbstständigkeitserklärung

Ich erkläre hiermit, dass ich diese Arbeit selbständig durchgeführt und keine anderen als die angegebene Quellen, Hilfsmittel und Hilfspersonen beigezogen habe. Alle Textstellen in der Arbeit, die wörtlich oder sinngemäss aus Quellen entnommen wurden, habe ich als solche gekennzeichnet.

Luis Wirth